PostProcessing_evaluator

An isolated workspace to build publication-ready postprocessing
Field inventory, SWC sampling, front-aware analysis — without touching the official pipeline

Nicole Flores

KAUST

May 13, 2026

Outline

Why this evaluator exists

Goal

Develop and validate postprocessing tools that extract time-dependent values along simulated neuronal geometries, and identify quantities + figures suitable for publication.

Problem with the old organization

The original outputs landed under `/scratch/flore0a/AnalysisResults/` without timestamps, without a manifest, and without a clean separation between raw and enriched data. Scripts were duplicated in two locations (`Postanalysis_swc/` and `Value_Extraction/`, byte-identical).

Approach

Stand up an evaluator workspace that **leaves the official pipeline untouched** and produces **timestamped, manifested** outputs to scratch. The official pipeline remains the ground-truth reference until the evaluator is validated.

Hard boundaries

Never modified

- `/project/.../plugins/Growth_Plugin/python_utils/PostProcessing`
  — the official Python postprocessing pipeline.
- `/project/.../apps/Neuronal_Process/PostProcessing`
  — the older app-side mirror.
- `/project/.../apps/Neuronal_Process/Scripts_lua/Model_only_final_steps.lua`
  — and any production Lua.
- `/scratch/floreOa/AnalysisResults`
  — historical postprocessing outputs (read-only reference).

Allowed

- Read everything above.
- Write only inside `/project/.../PostProcessing_evaluator/`

Layout (project side)

```
PostProcessing_evaluator/
  README.md
  copied_reference/                    (frozen, chmod 0444)
    README_PROVENANCE.md
    old_analysisresults_scripts/       (the 4 scripts that
                                        produced AnalysisResults)
  scripts/
    inspect_simulation_fields.py       (NEW; autodetect fields)
    reproduction/
      run_chain.py                     (NEW; orchestrator)
  sh_files/                            (login-node runners)
  slurm/                               (Shaheen jobs; srun+pvpython)
  presentation/
  configs/, reports/, tests/, logs/
```

Layout (scratch side)

```
/scratch/flore0a/PostProcessing_evaluator/
  inputs/    outputs/    figures/    tables/
  reports/                    (timestamped MD + CSV reports)
  runs/<TS>/                  (one inspection run per timestamp)
  reproduction_runs/<TS>/     (one full reproduction per timestamp)
  logs/                       (SLURM and login-node logs)
  temporary/
```

Every output directory carries a `YYYYMMDD_HHMMSS` stamp. The historical `AnalysisResults/` is never written to.

Stage 1: inspect_simulation_fields.py

What it does

Read-only inspection of one simulation directory.

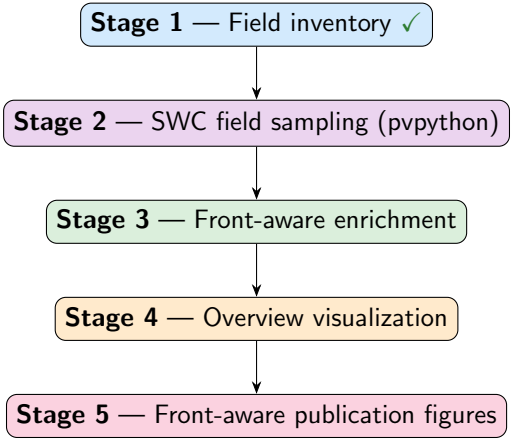
- Autodetects `Simulation.vtkhdf`, `merged/Output_t*.vtu`, `*.pvtu`.
- Enumerates timesteps and time values.
- For every point-data, cell-data, and field-data array: reports min, max, mean, std, n_{tuples}, and counts of NaN / Inf.
- Vector arrays are reported per-component plus magnitude.

Outputs

`/scratch/.../PostProcessing_evaluator/runs/<TS>/`

- `field_inventory.csv` — long format.
- `field_inventory.md` — per-step compact tables.
- `run_metadata.json` — command, executable, VTK version, hostname.

Workflow stages (recovered from AnalysisResults)



Stage 1 is new evaluator code. **Stages 2–5** are the 4 frozen scripts that produced `AnalysisResults` (verified via `IA_session_report_20260503_153249.md`).

Stages 6–8 (frozen reference, deferred)

Stage 6 — Morphometric extraction

`compute_morphometrics.py` (frozen). Per-frame morphometric features.

Stage 7 — Multivariate / statistical analysis

`analyze_morphometrics.py` (frozen). $D \times V$ parameter study, correlations, PCA, marginal effects, interaction terms.

Stage 8 — Mosaic / matrix visualization

`matrix.py` (frozen). Mosaic GIF assembly across cases.

Stages 6–8 need the full multi-case `Dataset_test*` tree, not a single case. AnalysisResults only contains D7.5_V3.

Publication-oriented analyses (planned)

1. **Front-aware kymographs** (Stage 5 `figA_*`) — tubulin / MAP2 / calcium / inhibitor along path-length over time.
2. **Growth-cone dynamics** (`figC`) — concentrations localized at the growth cone vs time; pair with elongation rate.
3. **Elongation rate vs concentration** (`figD`) — the coupling figure: dL/dt at lag-1 vs growth-cone concentration.
4. **L_max plateau** — D7.5_V3 plateaus at $L_{\max} \approx 3.05$ from frame ~ 5; not currently a figure but visible in `summary_report.txt`.
5. **Pixel-vs-clean skeleton agreement** — quantitative comparison of the two variants (currently visual only).
6. **Cross-case stability** — inspect every $D*\_V*$ in `Dataset_test{2,3,4}`; build per-field consistency reports.

Safety summary

Confirmed this session (read-only setup)

- Official PostProcessing modified: × **NO**.
- AnalysisResults modified: × **NO**.
- Production Lua modified: × **NO**.
- Existing results overwritten: × **NO**.
- Frozen reference (`copied_reference/`) `chmod 0444`.

Standing operational rules followed

- All SLURM scripts: `--mail-user=elsanicole.florespretell@kaust.edu.sa`, `--mail-type=ALL`.
- All compute-node writes to `/scratch/` only.
- All pvpython invocations under `srun --ntasks=1`.
- All outputs timestamped `YYYYMMDD_HHMMSS`.

Next steps (await user authorization)

1. Submit Stage 1 inspection on D7.5_V3 (`sbatch run_inspect_simulation_fields.slurm`).
2. Submit reproduction (`VARIANT=clean` first; then `pixel`).
3. Generate `reproduction_comparison_<TS>.md` comparing reproduced outputs vs `/scratch/flore0a/AnalysisResults`.
4. Extend to additional $D*\_V*$ cases.